

Lecture 3 (R lab):

R, RStudio and Quarto for Multivariate Data

STAT 481 — Multivariate Analysis

Sun, Aug 30, 2026 • 50 minutes • **bring your laptop — you type today**

Today's Overview

- **Thursday's data, in your hands** — the same 175 countries and eight indicators, loaded and cut about by you rather than by me.
- **This is the on-ramp for the whole course.** Every R chunk in the remaining 23 lectures assumes what happens in the next 50 minutes.
- `data.frame` versus `matrix` — two ways to hold **X**, and the reason Lecture 6 will need the second one.
- **Render your first Quarto report** — because R Lab 1 goes out today and is submitted that way.

Format: I demonstrate, then *now you*. Three checkpoints during the session; Worksheet 3 to take home. **No prior programming is assumed.**

Packages today

- `base` / `utils` / `stats` / `grDevices` [known] — `read.csv()` (`utils`), `apply()`, `tapply()` and `as.matrix()` (`base`), `sd()`, `cov()` and `cor()` (`stats`), and `pdf()` with `dev.off()` (`grDevices`), which write Step 7's figure to a file. All four start with R; nothing to install, nothing to load.
- `psych` [NEW] — `describe()`: a one-line numerical summary of every column at once. (We meet the rest of `psych` in Lecture 23, for factor analysis.)
- `quarto` [NEW] + `knitr` [NEW] — turn a `.qmd` file into a PDF. This is how every lab report is submitted.
- `sessioninfo` [NEW] — `session_info()`: the R and package versions, for the reproducibility appendix of your report.
- `renv` [NEW] — pins the exact package versions the course was built with, so a December result matches the August one. It works in the background.

All five [NEW] packages are already listed in `RProject/PACKAGES.md`. Run `source("R/setup.R")` and paste whatever install line it prints — **before Sunday**, not in class. *Data:* World Bank WDI, 2022. *Package:* Revelle, `psych`.

Learning objectives. By the end of this lab you can:

1. open the course project, and say why it replaces `setwd()`;
2. read a real CSV and check what arrived;
3. convert a `data.frame` to a `matrix` safely;
4. extract rows, columns and conditional subsets;
5. summarise every column with `apply()`, every group with `tapply()`;
6. render a `.qmd` to PDF from a fresh R session.

1 Why this session matters more than it looks

For the last two lectures I have been running R on the screen and you have been watching. From today you run it.

That is not a nicety. **Roughly two thirds of every quiz in this course asks you to read, write, predict, fix or interpret R code**, and the midterm and final are weighted the same way. The methods themselves — principal components, MANOVA, factor analysis — are two lines of code each.

Today involves *no new statistics whatsoever*, and no new data either. Every quantity we compute was defined on Thursday, on the same 175 countries. What changes is who types.

If you missed Thursday

Everything below is from Thursday, and Step 1 reloads the data from scratch, so nothing you missed blocks you. `name <- value` stores something under a name. `c("life_exp", "pop65")` bundles items into one. `round(x, 2)` shows two decimals and never changes the stored number. The **mean vector** $\bar{x}$ is the column averages, one per variable (`colMeans()`); the **covariance matrix** **S** is `cov()`; the **correlation matrix** **R** is `cor()`, whose entries r run from -1 to $+1$, and an r near ± 1 means two variables carry nearly the same information. Lecture 2, Sections 2–6; the wrap-up table on the last page gives the same map.

△! If something does not work, say so immediately

Being stuck silently for forty minutes is the worst possible outcome. Put your hand up the moment a command does not do what you expected. Nobody here has programmed before; that is the assumption the session is built on.

2 Step 1 — The project, and a real file

Find the course folder you downloaded from Blackboard and **double-click** `STAT481.Rproj`. RStudio opens. Now, in the console:

```
basename(getwd())      # which folder is R working in right now?
```

```
[1] "RProject"
```

`getwd()` is “get working directory” — the folder R treats as “here”. `basename()` strips off everything above it so we can print it without showing my whole hard disk. **Opening the .Rproj file is what set this**, and it is why the course project works on my machine and on yours without either of us editing anything.

△! Never write `setwd()` in anything you submit

It is tempting to fix a “file not found” error by adding `setwd("/Users/you/Desktop/stat481")`. That line makes your report run on exactly one computer — yours — and costs the reproducibility marks on every lab and on the project.

The rule: open the project, then use paths relative to it — a figure goes to “figures/L3.plot.pdf”, and today’s data comes from “data/worldhealth2022.csv”.

Now reload Thursday’s data — same two arguments, and this time you type them. `read.csv()` reads the plain text file in the project’s `data/` folder, and `row.names = 1` makes column 1 the row *labels* rather than a variable, which is why the ISO country codes are not a twelfth column (Lecture 2, §2).

```
wh <- read.csv("data/worldhealth2022.csv", row.names = 1)
dim(wh)
```

```
[1] 175  11
```

175 rows and 11 columns. `str()` says what is in them — `vec.len = 1` just shortens the preview so it fits this page:

```
str(wh, vec.len = 1)

'data.frame':   175 obs. of  11 variables:
 $ country      : chr  "Afghanistan" ...
 $ region       : chr  "Middle East, North Africa, Afghanistan & Pakistan" ...
 $ income       : chr  "Low income" ...
 $ life_exp     : num  65.6 ...
 $ infant_mort  : num  51.6 8.3 ...
 $ fertility    : num  4.93 ...
 $ pop65       : num  2.35 ...
 $ urban_pct    : num  25.4 ...
 $ gdp_pc       : num  357 ...
 $ health_exp_pct: num  23.1 ...
 $ internet_pct : num  15.9 ...
```

`str()` is the habit Lecture 1 asked for, and here it earns its keep. **Three columns are text** (`chr`) — the country's name, its World Bank region and its income group — and **eight are numbers** (`num`), in columns 4 to 11. On Thursday I handed you `wh[, 4:11]` without dwelling on why the range starts at 4. **That is why**, and Step 2 is about what happens if you forget it.

Notation

The eight numeric columns are the $n \times p$ data array **X** with $n = 175$, $p = 8$ — Lecture 2's **S** and **R** were computed from exactly these. The units table is on Lecture 2's second page; keep it beside you, because **three of the eight are percentages and one is dollars**, and Step 4 turns on that. Provenance: `data/worldhealth2022.CODEBOOK.md`, which you read before R Lab 1.

3 Step 2 — data.frame or matrix?

R has two rectangular containers, and the difference matters more in this course than in any other you will take.

```
class(wh)           # what kind of object is it?
is.matrix(wh)

[1] "data.frame"
[1] FALSE
```

`is.matrix()` answers a yes/no question with TRUE or FALSE.

Fact 3.1: The two containers, and when to use which

A **data frame** is a *list of columns*. Each column may have its own type — numbers in one, text in another. It is what `read.csv()` gives you and what almost all real data arrives as. Ours has three text columns and eight numeric ones.

A **matrix** is a single block of values, *all of the same type*. It is the object matrix algebra applies to.

Why we care in STAT 481: from Lecture 6 (Sun, Sep 13) onwards we multiply, transpose and invert **X**. Those operations need a *matrix*. `cov()` and `cor()` are polite and accept either; `%%` (matrix multiplication) is not. **Convert once, keep both objects around.**

So convert — and here is the trap. Try the obvious thing first:

```
as.matrix(wh)[1:3, c("country", "life_exp", "fertility")]
```

```
   country      life_exp fertility
AFG "Afghanistan" "65.617"  "4.932"
ALB "Albania"     "78.769"  "1.355"
DZA "Algeria"     "76.129"  "2.817"
```

Look at the quotation marks: 65.617 has become "65.617" — **text**.

⚠ as.matrix() on mixed data is silent and destructive

A matrix holds one type only, so the moment one column is text, *every* value is converted to text to match and the eight numeric columns are destroyed. There is no warning and no error; the object looks fine. Arithmetic afterwards fails, or worse, quietly misbehaves: as text, "110" sorts *before* "46", because it starts with a 1.

The cure: convert only the numeric columns. Run `str()` first — which we did — and check that every column you are converting is `int` or `num`.

Hence `wh[, 4:11]` from Thursday: select the numeric block first, convert *after*.

```
X <- as.matrix(wh[, 4:11]) # the eight numeric columns only
class(X)
dim(X)
```

```
[1] "matrix" "array"
[1] 175      8
```

`class(X)` answers with *two* words because in R a matrix is also an array; that is normal, not a second object. And `X` is 175×8 : the labels stayed behind in `wh`, where they belong. **Keep both.** We use `wh` whenever we need a country's name or group, and `X` whenever we do algebra.

4 Step 3 — Getting at the data

The indexing rule is one you already know from Lecture 1: **square brackets, row first, then column**, separated by a comma; an empty slot means "all". Two pieces of syntax go inside the brackets: `1:3` is shorthand for 1, 2, 3, and `c(...)` bundles items into one, so `c("life_exp", "fertility")` is a single column request naming two columns.

```
wh[1:3, c("country", "life_exp", "fertility")] # 3 rows, 3 columns
```

	country	life_exp	fertility
AFG	Afghanistan	65.617	4.932
ALB	Albania	78.769	1.355
DZA	Algeria	76.129	2.817

The row labels down the left are the country codes `read.csv()` took from column 1 — that is what `row.names = 1` bought us.

```
wh$life_exp[1:5] # that column, then its first 5 entries
```

```
[1] 65.617 78.769 76.129 84.016 64.246
```

Note the missing comma: `wh$life_exp` is already a single column, so it takes one index. The row-comma-column rule applies to the whole table, not to a column pulled out of it.

The really useful move is subsetting by a *condition*. Put a logical test where the row index goes:

```
wh[wh$life_exp > 84, c("country", "life_exp", "pop65")]
```

	country	life_exp	pop65
AND	Andorra	84.016	14.969
MCO	Monaco	85.746	37.061
SMR	San Marino	85.708	21.024

Three countries pass. Read `wh$life_exp > 84` as a question asked of all 175 countries at once, returning 175 TRUE/FALSE answers; R then keeps the rows where the answer was TRUE. Notice all three are tiny, rich and European — hold that thought until Step 7.

Now you — Checkpoint 1 — getting at the data

Work in the console. Write down what R printed for each part.

1. Extract the row for **Qatar** — its code is "QAT" — with all eleven columns.
2. Extract the `life_exp` and `internet_pct` columns for the *first four* countries.
3. List the countries with fertility above 5 births per woman, showing only `country` and `fertility`. How many are there?
4. **Predict before you run:** what will `dim(wh[wh$life_exp > 84, ])` print?

5 Step 4 — `apply()`, and a question it answers wrongly

You could compute eight means with eight commands. Instead, tell R to apply one function along one *margin* of the matrix. Read the next line from the inside out: `apply()` takes three things — the matrix, the margin, and *the name of a function*, here `mean` with no brackets after it, because you are handing R the function rather than calling it. `round(..., 2)` then tidies the eight answers for printing.

```
round(apply(X, 2, mean), 2)      # MARGIN = 2 -> work down each COLUMN
```

<code>life_exp</code>	<code>infant_mort</code>	<code>fertility</code>	<code>pop65</code>	<code>urban_pct</code>
72.53	19.98	2.47	9.74	61.63
<code>gdp_pc</code>	<code>health_exp_pct</code>	<code>internet_pct</code>		
18450.92	6.82	69.56		

There is the mean vector $\bar{\mathbf{x}}$ of Lecture 2, on eight variables: the average country in 2022 lives 72.5 years, has 2.47 children, and is 61.6% urban. The second argument is the **margin**: 1 means *rows*, 2 means *columns* — the same order as `dim()`, rows before columns, which is how to remember it. `colMeans(X)` gives the identical answer; `apply()` is the general tool, `colMeans()` the fast special case.

Now the spreads:

```
round(apply(X, 2, sd), 2)      # the eight standard deviations
```

<code>life_exp</code>	<code>infant_mort</code>	<code>fertility</code>	<code>pop65</code>	<code>urban_pct</code>
8.20	22.75	1.26	7.14	22.08
<code>gdp_pc</code>	<code>health_exp_pct</code>	<code>internet_pct</code>		
28313.17	3.27	25.47		

Self-check (1 minute)

Which of the eight variables *varies the most*? Answer before you read on.

If you answered `gdp_pc`: 28,313.17 is bigger than 8.20, but it is 28,313.17 *dollars* against 8.20 *years*. A standard deviation carries the units of its own variable (Lecture 2, §5), so the two cannot be compared, and the question has no answer as asked.

What that number *does* decide, you already met on Thursday. Look again at the variances — the diagonal of **S** — but this time reached through `apply()`:

```
round(diag(cov(X)), 1)
```

<code>life_exp</code>	<code>infant_mort</code>	<code>fertility</code>	<code>pop65</code>	<code>urban_pct</code>
67.3	517.7	1.6	51.0	487.4
<code>gdp_pc</code>	<code>health_exp_pct</code>	<code>internet_pct</code>		
801635374.1	10.7	648.6		

Add them up and ask what share each one contributes:

```
round(100 * diag(cov(X)) / sum(diag(cov(X))), 3) # percent of total
```

life_exp	infant_mort	fertility	pop65	urban_pct
0	0	0	0	0
gdp_pc	health_exp_pct	internet_pct		
100	0	0		

This is Thursday's headline in a different wrapper: **GDP per capita is 100% of the total variance to three decimal places**, which is why Lecture 2 concluded that **S** cannot answer a question about these eight variables and **R** can. The other seven are not literally zero — ask for more digits and `internet_pct` is 8.09×10^{-5} percent — but R rounds them away.

What is new today is what that does to *your* code.

△! MARGIN = 1 is legal, and here it is nonsense

Watch what happens if you get the margin wrong (the `>` below is R's own prompt, not something you type):

```
> round(head(apply(X, 1, mean), 4), 1)
  AFG   ALB   DZA   AND
68.3 1001.1 652.3 5338.4
```

R returned the average *within each country*, across all eight variables — and each is essentially that country's GDP divided by eight. Andorra's eight values sum to 42,707, of which **GDP per capita is 42,414**; the other seven contribute 0.7% of the answer. **That is Thursday's total-variance result — the trace of S — showing up in your own output**, in a plain sum instead of a sum of squares — the same variable swamping the same seven companions. Averaging dollars with years, children and percentages is not an error R can detect.

Say out loud which margin you want before you type: “one number per variable” is `MARGIN = 2`; “one number per country” is `MARGIN = 1`, and is almost always wrong when the columns have different units.

Now you — Checkpoint 2 — `apply()` and the margin

Part 1 in class; parts 2 and 3 at home, checked at the start of Lecture 4.

1. Use `apply()` to get the *median* of each of the eight variables. Two of them have a median far below their mean, and one has a median well *above* it. Name all three and say what each tells you about the shape of that variable.
2. Compute `apply(X, 2, range)`. What are the *dimensions* of the result, and why is it not a plain vector?
3. **Predict, then check:** how many numbers does `apply(X, 1, mean)` return — 8 or 175? Is the answer useful?

6 Step 5 — One line, and what it catches

`apply()` is general. For the specific job of “describe every column”, there is a purpose-built function. `psych::describe` means “the `describe` that lives in `psych`”; the double colon names a function's home package, and after `library(psych)` you need only type `describe()`.

```
library(psych)
describe(wh[, c("life_exp", "gdp_pc", "internet_pct")])
```

	vars	n	mean	sd	median	trimmed	mad	min
life_exp	1	175	72.53	8.20	74.10	73.08	7.74	18.82
gdp_pc	2	175	18450.92	28313.17	7068.22	12455.25	8689.07	301.83
internet_pct	3	175	69.56	25.47	78.63	72.41	20.29	7.14
	max	range	skew	kurtosis	se			
life_exp	85.75	66.93	-1.78	8.87	0.62			
gdp_pc	226052.06	225750.23	3.43	17.15	2140.27			
internet_pct	100.00	92.86	-0.88	-0.43	1.93			

The output is too wide for the console, so R wraps it — normal, and worth seeing once. *Where* it breaks follows your console width, so your break may fall a column from the one printed here. Ignore **trimmed**, **mad** and **se**; **skew** is positive when the long tail is on the right.

Three columns, three shapes, and you read them by comparing **mean** with **median**. **gdp_pc**: mean 18,451 against median 7,068 and skew 3.43 — **a long right tail**, a few very rich countries pulling the average above the typical country. **internet_pct** is the reverse: mean 69.6, median 78.6, skew -0.88 — **a left tail**, most of the world bunched near the top with a straggle below.

Now look at **min** for **life_exp**.

Self-check (1 minute)

The lowest life expectancy in the world, in this table, is **18.82 years**. Is that possible? Take ten seconds before reading on.

It is not. No country has ever had a life expectancy of 18.8 years. Find it:

```
wh[wh$life_exp < 40, c("country", "life_exp", "infant_mort")]
```

	country	life_exp	infant_mort
CAF	Central African Republic	18.818	206.8

Both numbers are impossible: 206.8 infant deaths per 1,000 live births is one baby in five, well over double the highest rate anywhere else on earth (Somalia, 89.2). The country's other six indicators are unremarkable for the region — fertility 6.02, GDP per capita \$467, internet use 13.8% — so this is not a country in catastrophe. It is **two broken cells**.

Fact 6.1: Provenance is not validity

I checked the World Bank's database directly. **These are the values the World Bank publishes** for the Central African Republic in 2022, and its own series is unstable around them: life expectancy reads 50.6 in 2020, 40.3 in 2021, 18.8 in 2022 and 57.4 in 2023. Our file copied the source faithfully, so the fault is upstream — and it is still there.

An authoritative source, a correct import, and a number that cannot be true. The only reason we know is that we spent one line on a summary before doing anything else. **That is the habit this lecture exists to install.** We leave the file as it is: a data set with a known flaw in a known row is more useful to you than a quietly patched one.

One more turn. Lecture 2 found that two variables can be almost the same variable; here that pair is life expectancy and infant mortality, and the sign is *negative*. What does the broken row do to it?

```
ok <- wh$life_exp > 40 # everything except the bad row
round(c(all = cor(wh$life_exp, wh$infant_mort),
      ok = cor(wh$life_exp[ok], wh$infant_mort[ok])), 3)
```

all	ok
-0.917	-0.894

⚠ A bad point can make your result look *better*

Dropping the impossible row moves the correlation from -0.917 to -0.894 — **weaker**. The corrupt point sits far out along the same downward trend, and a far-out point in the direction

△! A bad point can make your result look *better* (continued)

you expected drags $|r|$ up. So “the correlation is strong” is not a check that the data is **sound** — a strong r is exactly what a well-placed bad value produces. Summaries first, picture second, correlation third.

With that settled, here is **R** for four of the eight — the 1.000 down the diagonal is each variable’s correlation with itself:

```
round(cor(wh[, c("life_exp", "infant_mort", "fertility",
                "internet_pct")]), 3)
```

	life_exp	infant_mort	fertility	internet_pct
life_exp	1.000	-0.917	-0.809	0.774
infant_mort	-0.917	1.000	0.779	-0.735
fertility	-0.809	0.779	1.000	-0.841
internet_pct	0.774	-0.735	-0.841	1.000

Every entry has a readable sign: longer lives go with fewer infant deaths (−0.917), fewer births per woman (−0.809) and more internet use (+0.774). Thursday’s 8×8 version of this matrix is the same numbers in full; **what today adds is that you can now cut four variables out of it yourself** — and that the −0.917 Lecture 2 called the strongest pair was, as Step 5 just showed, propped up slightly by a row that cannot be true.

7 Step 6 — Summaries *by group*

Every summary so far has treated all 175 countries as one lump. But two of the three text columns we set aside in Step 2 are group labels, and they are the most interesting thing in the file. **tapply()** splits a variable by a label and applies a function to each piece — the same function-handed-over idea as **apply()**, with a grouping column in the middle:

```
round(tapply(wh$life_exp, wh$income, mean), 1)
```

	High income	Low income	Lower middle income
	79.0	60.2	67.5
Upper middle income	73.4		

△! The order R printed is alphabetical, not economic

“High” comes before “Low” in the dictionary, so the poorest group prints *second* and the sequence reads as nonsense. R sorted the labels as text because that is all they are to R. **Reorder them yourself**, using exactly the **c(...)** indexing from Step 3.

Here it is applied to a named result rather than to a table:

```
econ <- c("Low income", "Lower middle income",
         "Upper middle income", "High income")
round(tapply(wh$life_exp, wh$income, mean)[econ], 1)
```

	Low income	Lower middle income	Upper middle income
	60.2	67.5	73.4
High income	79.0		

Now it reads: 60.2 → 67.5 → 73.4 → 79.0, a clean nineteen-year climb. **Seven of the eight indicators do this** — each one rises, or falls, without a single step out of place across the four groups. Exactly one does not:

```
round(tapply(wh$health_exp_pct, wh$income, mean)[econ], 2)
```

	Low income	Lower middle income	Upper middle income
	6.82	5.29	6.52
High income	8.11		

Fact 7.1: The one indicator that does not follow income

Health spending as a share of GDP falls from 6.82% to 5.29% between the two poorest groups, then climbs to 8.11%. **The world's poorest countries devote a larger share of their economy to health than the lower-middle-income countries above them.**

Not a paradox and not an error: a share is a ratio, and here the *denominator* moves fastest. 6.82% of the low-income average GDP per capita (\$723) is about \$49 a person; 5.29% of the lower-middle average (\$2,448) is about \$130 — more than twice the money for a smaller share.

The transferable point: a variable measured as a proportion of something else can move for reasons that have nothing to do with what you think you are measuring — and a three-line group summary was enough to find the one column in eight that behaves differently from its neighbours.

One caution before you use `tapply()` on anything else: always run `table()` on the grouping column first — it counts the rows per label. These four groups hold 19, 42, 52 and 62 countries, so every mean above rests on something. Group by **region** instead and North America has *two* members — a “regional mean” that is one country with another attached.

8 Step 7 — The country no single variable flags

In Step 3 the three longest-lived countries were Andorra, Monaco and San Marino, and every one of them had 15% or more of its population aged 65 or over — Monaco fully 37%. That is the usual pattern: people who live a long time accumulate as old people, and across all 175 countries life expectancy and `pop65` correlate +0.66 — +0.72 once Step 5's broken row is dropped.

Here is that relationship. *One point is nothing like the others* — find it before you read the next paragraph.



Life expectancy against the percentage aged 65 and over, 175 countries, 2022. The lone point at the lower right lives as long as Europe and has almost no elderly at all. (The point far to the left, alone at 18.8 years, is the broken row from Step 5 — a second reason to plot before you model.)

It is Qatar. Qatar is 19th in the world on life expectancy, and the eighteen countries above it have between 12.7% and 37.1% of their population aged 65 or over. Qatar has **1.39%** — the top ten, then the rest:

	life exp	pop65		life exp	pop65
Monaco	85.75	37.06	Switzerland	83.61	19.22
San Marino	85.71	21.02	Australia	83.20	17.08
Andorra	84.02	14.97	Spain	83.13	20.19
Japan	84.00	29.40	Sweden	83.06	20.34
Luxembourg	82.95	14.84	Singapore	82.89	12.69
ranks 11–18 (Italy, Korea, Ireland, Norway, Malta, France, Iceland, New Zealand): <i>pop65</i> 15.11 to 23.93					
Qatar	81.86	1.39	— 19th on life expectancy, last on <i>pop65</i> in the world		

Now the question this lecture has been building towards: is Qatar unusual on any *single* variable? Standardise its row — subtract each variable’s mean and divide by that variable’s standard deviation, putting all eight on the common scale “how many standard deviations from average”:

```
zq <- (X["QAT", ] - apply(X, 2, mean)) / apply(X, 2, sd)
round(zq, 2)
```

life_exp	infant_mort	fertility	pop65	urban_pct
1.14	-0.66	-0.58	-1.17	1.70
gdp_pc	health_exp_pct	internet_pct		
2.48	-1.42	1.12		

One line, three things worth naming. `X["QAT", ]` pulls out one row — eight numbers. `apply(X, 2, mean)` is eight numbers too, so R subtracts them *position by position*, life expectancy from life expectancy, then divides the same way; you never wrote a loop. And the result is unitless, so for the first time today the eight numbers are comparable with each other.

Fact 8.1: Unusual in no variable, unusual in the pair

Qatar’s largest deviation is `gdp_pc` at 2.48 standard deviations — noticeable, but 16 countries have a bigger single deviation, so on that measure Qatar ranks only **17th of 175**. Screen the eight variables one at a time and Qatar is unremarkable.

Yet its *combination* is unlike any other country on earth: +1.14 on life expectancy *and* -1.17 on the share aged 65+, when those two correlate +0.66. (The reason is a population dominated by working-age migrant labour: the elderly are not absent because people die young — they are simply not there.)

This is the case for multivariate analysis in one example. Eight one-variable screens miss Qatar completely; a method that looks at the variables *together* finds it at once — and building that method is what **Lecture 5, Thursday** is for.

9 Step 8 — Quarto: your first reproducible report

Everything you submit in this course is a **Quarto** document: one file holding your writing, your code and R’s output, which renders to a PDF. This is the step that must work before you leave.

In RStudio: **File** → **New File** → **Quarto Document**, give it a title, choose PDF, and Create. Delete everything the template gives you and type this:

```
---
title: "STAT 481 --- my first report"
author: "Your Name, Your ID"
format: pdf
---

## What this data set is

```{r}
wh <- read.csv("data/worldhealth2022.csv", row.names = 1)
dim(wh)
```

The file holds 175 countries and 11 columns, of which 8 are numeric.

## Two variables that are almost one variable

```{r}
round(cor(wh$life_exp, wh$infant_mort), 3)
```

The sign is negative: where more infants die, people live less long.

## Reproducibility

```{r}
sessioninfo::session_info()$platform$version
```
```

Three kinds of line there. The `---` fences at the top hold the document settings (title, author, output format). A line starting with `##` is a section heading. Triple backticks followed by `{r}` open a **code chunk** and triple backticks close it: inside is R, outside is your writing.

Save it as `firstreport.qmd` inside the project folder, and click **Render**.

Fact 9.1: What just happened

Quarto ran your R code in a **brand-new R session**, captured whatever it printed, and wove code, output and prose into one PDF. That fresh session is the point: it is why the document proves the result rather than merely asserting it.

It is also why a report that leans on an object you made in the console last Tuesday fails: the fresh session does not have it. **Notice that the report reads the CSV itself** — it does not assume `wh` exists.

Two details in the chunks above: `cor()` given *two columns* returns a single number rather than a matrix; and `$platform$version` reaches two levels down inside what `session.info()` hands back.

Now you — Checkpoint 3 — render your first report (do not leave without this)

1. Create `firstreport.qmd` as above and **Render** it to PDF.
2. Add a fourth section that computes, with `colMeans()`, the mean vector of the four *demographic* variables (`life_exp`, `infant_mort`, `fertility`, `pop65`), and one sentence saying what it tells you. Render again.
3. Now the real test: **Session** → **Restart R**, then Render once more *without running any chunk by hand*. If it still works, your report is reproducible. If it does not, the error tells you exactly which object you forgot to create inside the document.

Now you — Checkpoint 3 — render your first report (do not leave without this) (continued)

Show me the rendered PDF before you leave. If the PDF step fails because a $\text{\LaTeX}$ engine is missing, render to HTML instead so the rest of the checkpoint still works, and see me about PDF before you leave — not the night before Sep 6.

△! The three ways students lose reproducibility marks

1. The document does not `library()` its own packages. **2.** The document does not read its own data. **3.** The document contains `setwd()` or an absolute path.

All three have the same test and the same cure: **restart R and render**, before every submission.

Wrap-up

| Concept | R function | Where you learned it |
|---------------------------------------|---|---|
| The $n \times p$ data array X | <code>dim()</code> , <code>str()</code> , <code>head()</code> | Lecture 1, Tue Aug 25 |
| Reading a real data file | <code>read.csv()</code> | Lecture 2; typed by you today |
| One row, one column, one cell | <code>[i, k]</code> , <code>\$</code> , <code>[cond,]</code> | Lectures 1 and 3 |
| Data frame vs. matrix | <code>class()</code> , <code>as.matrix()</code> | Lecture 3, today |
| Sample mean vector $\bar{\mathbf{x}}$ | <code>colMeans()</code> , <code>apply(X, 2, mean)</code> | <code>colMeans()</code> Lecture 2; <code>apply()</code> today |
| Sample covariance matrix S | <code>cov()</code> , <code>diag(cov())</code> | Lecture 2; the diagonal today |
| Sample correlation matrix R | <code>cor()</code> , <code>corrplot()</code> | Lecture 2 |
| Generalized variance $ \mathbf{S} $ | <code>det(cov())</code> | Lecture 2 |
| Column-by-column summary | <code>psych::describe()</code> | Lecture 3, today |
| Summary by group | <code>tapply()</code> | Lecture 3, today |
| Standardising a row | <code>(x - mean) / sd</code> | Lecture 3, today |
| Saving a figure | <code>pdf()</code> ... <code>dev.off()</code> | Lecture 3, today |
| A reproducible report | Quarto <code>.qmd</code> → Render | Lecture 3, today |

Every later lab extends this table — keep it.

Summary of Lecture 3

- **Open the project, never `setwd()`.** Relative paths only.
- **Data frame** for real data, **matrix** for algebra — and convert *only the numeric columns*, or every number silently becomes text.
- `apply(X, 2, f)` works *down columns* — one number per variable. `MARGIN = 1` works across rows and, when the columns have different units, returns something with no meaning.
- **A standard deviation carries its own variable's units**, so “which varies most” is not a question raw numbers can answer — and one variable in dollars can own 100% of the total variance.
- **Summarise before you analyse**, and a strong correlation is not evidence the data is clean.
- **Restart R and render** before you submit anything. Every time.
- **The habit to keep:** `dim()`, `str()`, `head()` on arrival; `describe()` next; `cov()` and `cor()` after that; a picture before any conclusion.

Next lecture (Tue, Sep 1): *Seeing Multivariate Data* — scatterplot matrices, stars and Chernoff faces, and the honest limits of each. Back to lecture format, but with more R than either of the first two.

Summary of Lecture 3 (continued)

Announcements:

- **R Lab 1** is out today and is due **Sun, Sep 6**. Submit the .qmd *and* the rendered PDF on Blackboard.
- **Practice Sheet 1** is the whole session on **Sun, Sep 6**.
- **Sample Quiz 1** is on Blackboard; we correct it in class **Tue, Sep 8**.
- **Quiz 1: Thu, Sep 10**, covering Lectures 1–5. You will not run R in the quiz — all output is printed on the paper.
- Add/Drop closed on Thursday.

Saving a figure from code (*read at home — Worksheet 3 needs it*)

Never export a plot by right-clicking it; a figure you cannot regenerate is a figure you cannot trust. Open a file device, draw, close it. This is the exact code that produced the figure in Step 7:

```
pdf("figures/L3_life_pop65.pdf", width = 6, height = 4.2)
par(mar = c(4.2, 4.2, 0.6, 0.6)) # margins in lines: b, l, t, r
plot(wh$life_exp, wh$pop65, pch = 21, bg = "grey80", col = "grey30",
      xlab = "life expectancy at birth (years)",
      ylab = "population aged 65+ (%)")
dev.off()
```

The path is **relative**, so it runs anywhere; `pch = 21` draws a circle with a fill (`bg`) and a rim (`col`), which survives a black-and-white printer; and `dev.off()` is not optional — until you run it the file is open and usually empty.

Take-home — Worksheet 3

Worksheet 3 is on Blackboard and it is your work for this lecture. It is not collected and not graded; we go through it together at the start of **Lecture 4, Tue Sep 1** — bring it.

It takes the one claim Step 7 asked you to accept — *that Qatar is unusual in no single variable and unusual in the pair* — and makes you measure it, in sixteen short questions on this same file. You will find out exactly which countries a standard one-variable-at-a-time screen does catch, why lowering its threshold does not rescue it, and what happens when you build one number out of two columns instead of eight numbers out of one column each.

It is also the on-ramp to Thursday. Lecture 5 opens on the number Worksheet 3 ends with.